Desenho de Algoritmos

Turma 5

Dário Amaral
Eduardo Santos
Gonçalo Pinto

Enquadramento do problema

Contexto:

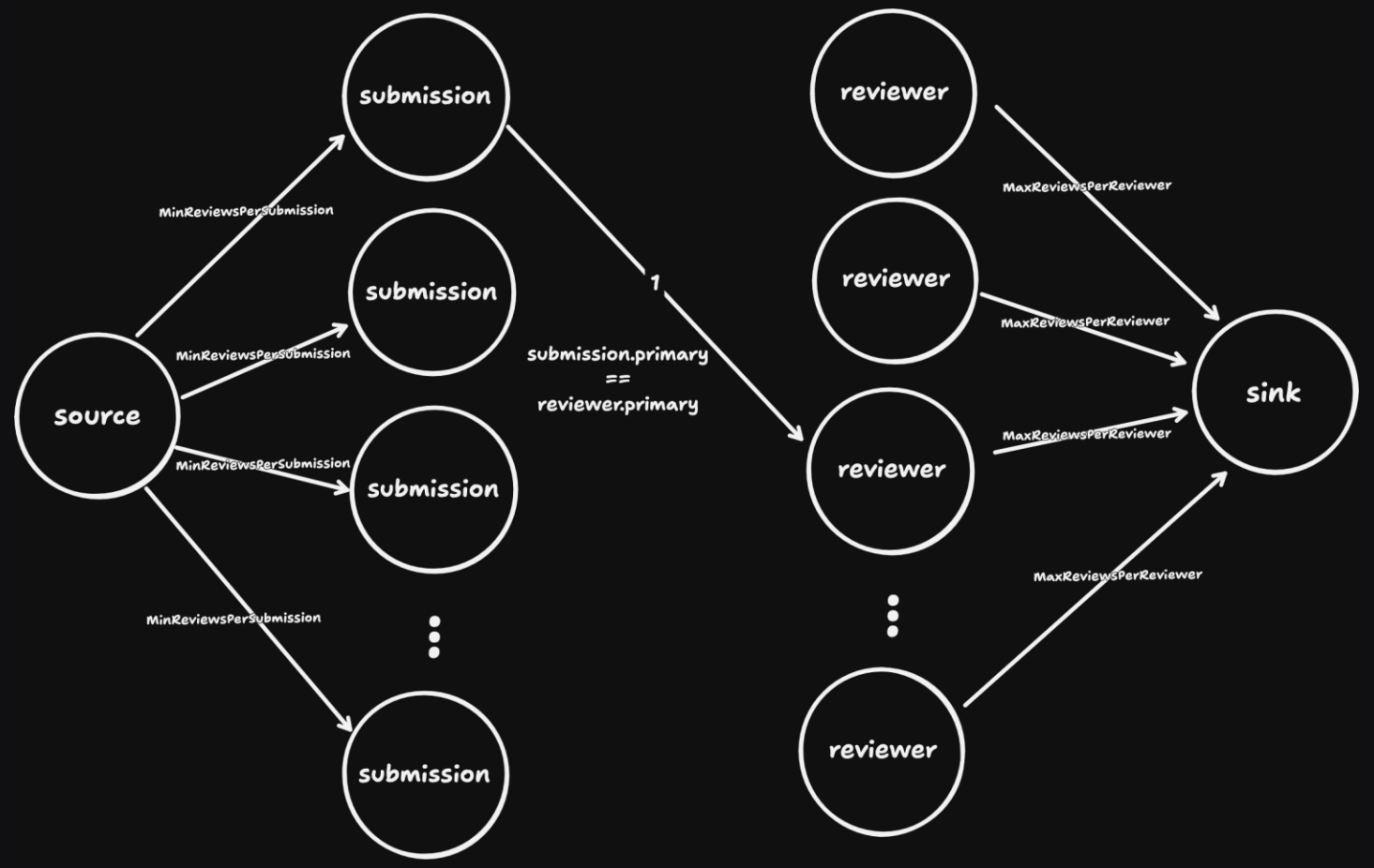
- Uma conferência científica recebe N paper submissions
- Cada submission tem um tópico primário (e opcionalmente secundário)
- Existem M reviewers, cada um com expertise num ou dois tópicos

Restrições:

- Cada submission precisa de ≥ MinReviewsPerSubmission reviews
- Cada reviewer faz ≤ MaxReviewsPerReviewer reviews
- Um reviewer só pode rever uma submission se os tópicos forem compatíveis

Objetivo: Encontrar uma atribuição válida - ou reportar inviabilidade

Enquadramento do problema



Exemplo do grafo quando GenerateAssignments = 1

Arquitetura da solução

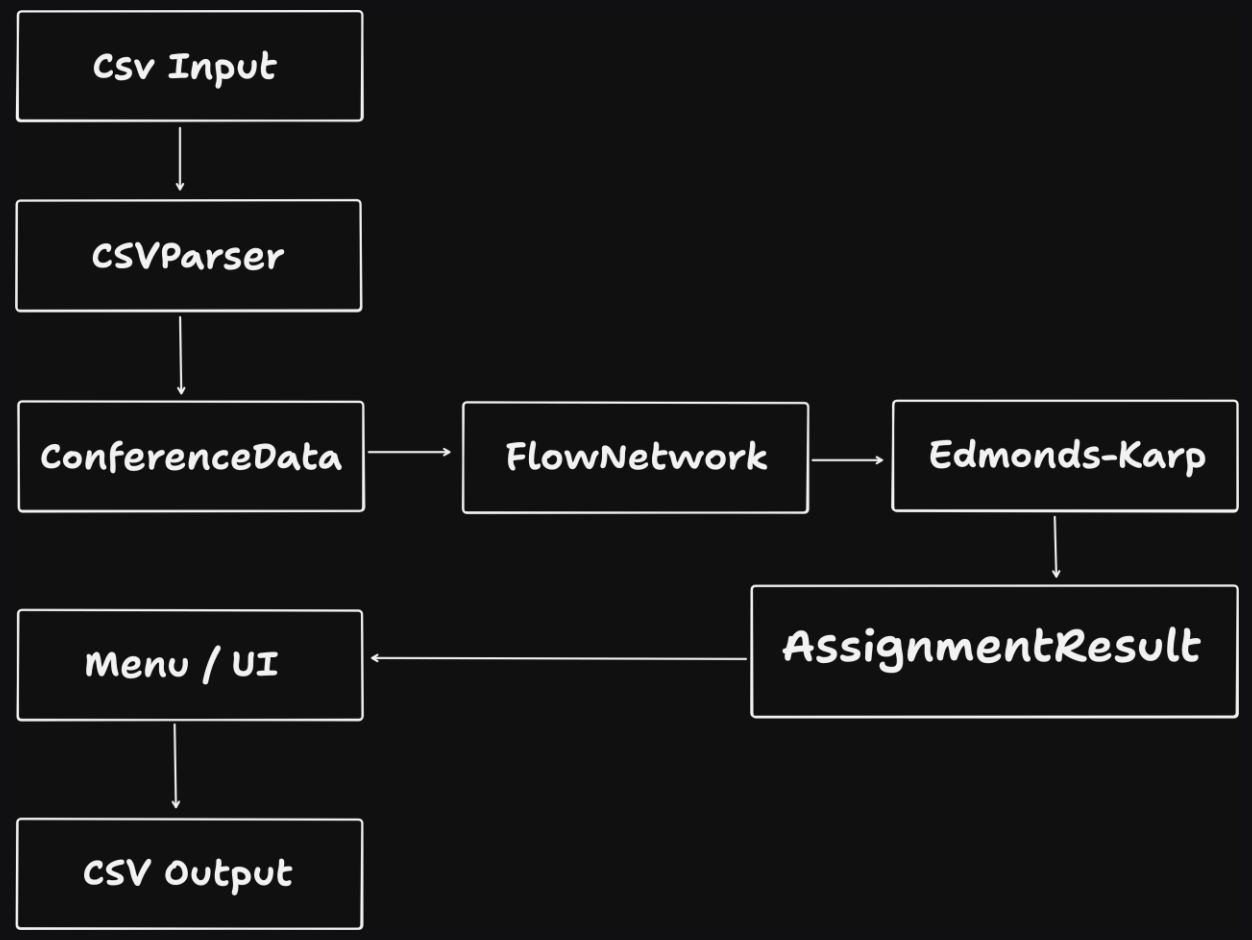


Diagrama de fluxo

```
src/
├── model/     -> Submission, Reviewer, Parameters, Control
├── parser/    -> CSVParser
├── algorithm/ -> FlowNetwork
├── ui/        -> Menu
├── graph/
├── output/
└── main.cpp
```

Estrutura do projeto

Estrutura do input

Ficheiro CSV com 4 secções:

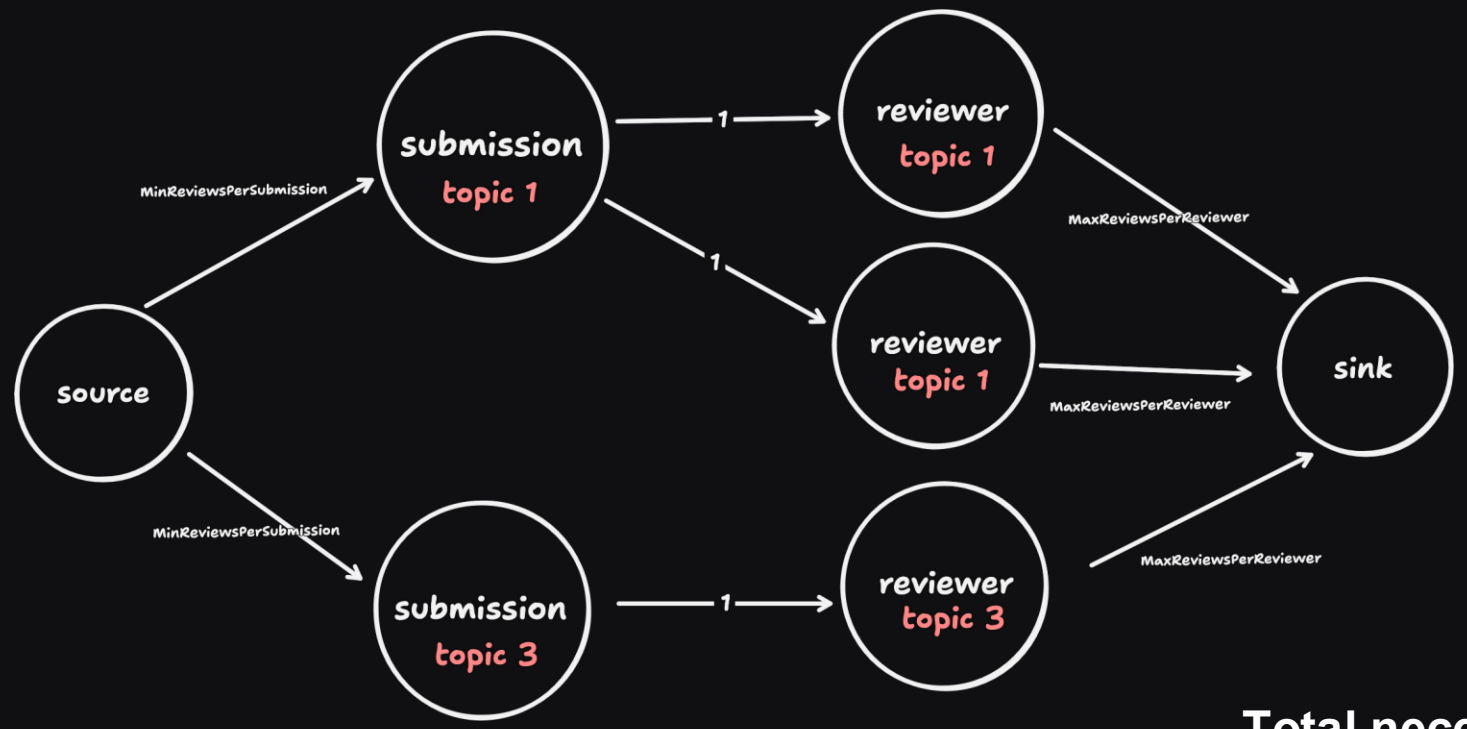
#Submissions
-> Id, Title, Authors, Email, Primary, Secondary

#Reviewers
-> Id, Name, Email, Primary, Secondary

#Parameters
-> MinReviewsPerSubmission
-> MaxReviewsPerReviewer
-> Pesos de matching (Primary/Secondary)

#Control
-> GenerateAssignments (0-3)
-> RiskAnalysis (0, 1, K)
-> OutputFileName

Formulação como problema de fluxo



Condição de Sucesso:

Total necessário = Σ MinReviewsPerSubmission

Se maxFlow == Total -> Atribuição VIÁVEL

Se maxFlow < Total -> Atribuição INVIÁVEL

Compatibilidade entre Submissions e Reviewers

Valor	Condição de Compatibilidade
1 (base)	submission.primary == reviewer.primary
2	submission.primary/secondary == reviewer.primary
3 (geral)	Qualquer combinação primary/secundary

Algoritmo Edmonds-Karp

```
Edmonds-Karp (Ford-Fulkerson com BFS):

function solve(graph, source, sink):
    maxFlow = 0
    inicializar flow = 0 em todas as arestas

    enquanto BFS encontrar caminho source → sink:
        bottleneck = min(cap - flow) ao longo do caminho
para cada aresta no caminho:
            flow[aresta] += bottleneck
            flow[aresta_reversa] -= bottleneck
        maxFlow += bottleneck
    return maxFlow
```

Complexidade:
$O(V \times E^2)$

Extração do Assignment

Formato do output exigido

```
#SubmissionId,ReviewerId,Match
31, 1, 3
87, 2, 1

#ReviewerId,SubmissionId,Match
1, 31, 3
2, 87, 1
#Total: 2

#SubmissionId,Domain,MissingReviews
31,3,5
87,1,3
```

Depois de correr Edmonds-Karp:

Para cada aresta (Sub_i → Rev_j):
  se flow == 1:
    → Rev_j está atribuído a Sub_i

Para verificar inviabilidade:
  reviews_obtidas[i] = Σ flow nas arestas saindo de Sub_i
  se reviews_obtidas[i] < MinReviews:
    → Sub_i tem (MinReviews - reviews_obtidas[i]) reviews em falta

Demo: Exemplo com dataset1

Input (dataset1.csv):

- 2 **submissions** (topics: 3, 1)
- 2 **reviewers** (expertises: 1, 1)
- MinReviewsPerSubmission = 5
- MaxReviewsPerReviewer = 4
- GenerateAssignments = 1

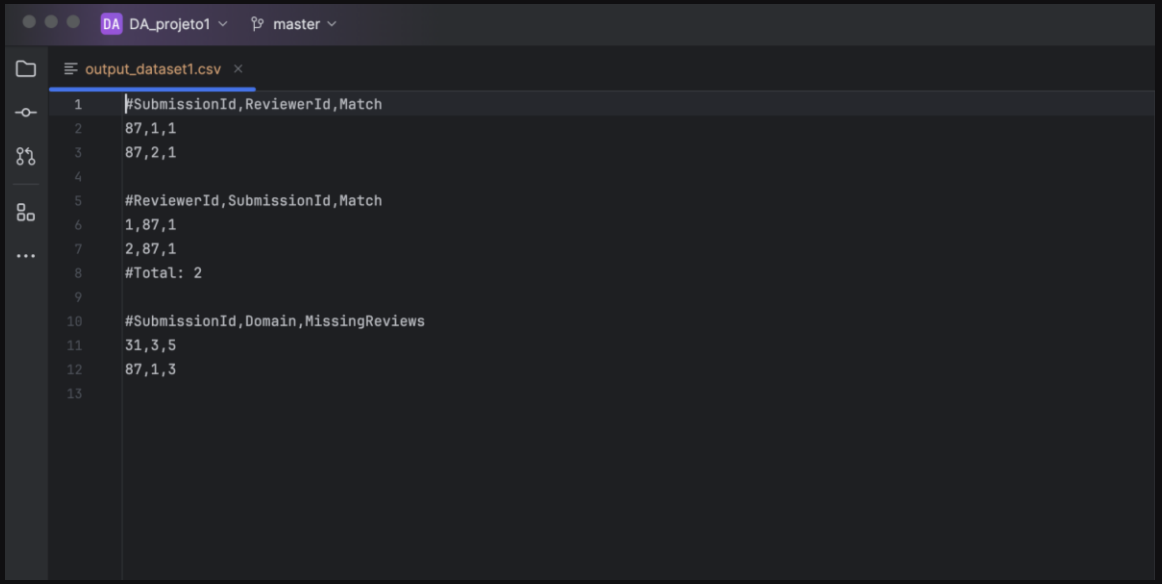
Fluxo máximo atingido: 2 / 10

Output:

#SubmissionId,Domain,MissingReviews

31, 3, 5 ← nenhum reviewer tem expertise 3!

87, 1, 3 ← só 2 reviewers, cada um faz max 4



Funcionalidades implementadas

Funcionalidade	Estado
Menu interativo	✅
Leitura e parsing de datasets	✅
Listagem de submission	✅
Listagem de reviewers	✅
Visualização de parâmetros	✅
Geração de assignments (mode 1)	✅
Deteção de inviabilidade	✅
Execução em batch mode	✅
Risk analysis = 1	✅
Documentação Doxygen	✅

Risk Analysis = 1

Pergunta: existe um reviewer cuja ausência torna o assignment impossível?

Abordagem (brute force):

para cada reviewer r:
  remover r do grafo (capacidades a 0)
  recalcular max-flow
  se maxFlow < Total necessário:
    → r é um reviewer CRÍTICO ⚠️
  restaurar r

Output:
#Risk Analysis: 1
1, 2 ← reviewers críticos, ordenados

Complexidade:
$O(M \times V \times E^2)$

M - nº de reviewers

T2.3: Risk Analysis = K (abordagem teórica)

Naive: testar todos os C(M, K) subconjuntos de K reviewers
→ Complexidade: $O(C(M,K) \times V \times E^2)$ — exponencial em K

Mais eficiente: node splitting + min vertex cut
→ Cada reviewer r dividido em r_in → r_out com cap=1
→ Encontrar min vertex cut de tamanho ≤ K
→ Complexidade: $O(K \times M \times V \times E^2)$

Exemplo não trivial:
3 submissions, 4 reviewers
Remover qualquer 1 → assignment OK
Remover reviewers {2, 3} → assignment FALHA
→ K=2 é o threshold crítico

T2.4: Formulação geral

Considerar domínios primários e secundários:

Submission	Reviewer	Match
Primary (tp)	Primary (rp)	tp == rp
Primary (tp)	Secondary (rs)	tp == rs
Secondary (ts)	Primary (rp)	ts == rp
Secondary (ts)	Secondary (rs)	ts == rs

Abordagem: mesma rede de fluxo, mais arestas
→ Adicionar aresta (Sub_i → Rev_j, cap=1) para cada combinação válida
→ Complexidade aumenta mas continua polinomial
→ Grafo pode ter até 4× mais arestas que o caso base

Complexidade das componentes principais

Componente	Complexidade	Justificação
Parsing do input	O(L)	L = linhas do ficheiro
Construção do grafo	O(N x M)	Arestas Sub -> Rev
Edmonds-Karp	$O(V \times E^2)$	BFS x arestas ao quadrado
Risk Analysis = 1	$O(M \times V \times E^2)$	Repete max-flow M vezes
Risk Analysis = K (naive)	$O(C(M,K) \times V \times E^2)$	Exponencial em K

No nosso grafo:
V = N + M + 2 (submissions + reviewers + source + sink)
E = N + M + compatibilidades

Conclusões

- ✅ Problema modelado com sucesso como rede de fluxo bipartida
- ✅ Parser robusto com validação de dados e IDs duplicados
- ✅ Edmonds-Karp implementado sobre a estrutura de grafos das aulas TP
- ✅ Assignment gerado e inviabilidade reportada com detalhe
- ✅ Risk Analysis = 1 implementado e testado
- ✅ Abordagens teóricas desenvolvidas para K > 1 e formulação geral
- ✅ Documentação completa com Doxygen

"A formulação como Max-Flow transforma um problema
de atribuição complexo numa solução polinomial elegante."